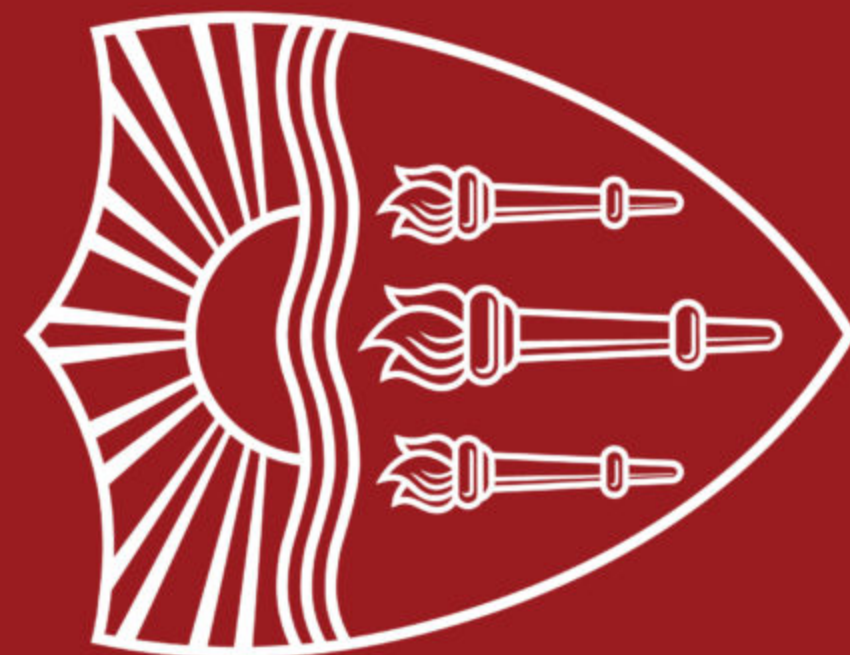




Lecture 12:

Tokenization and Generation Algorithms

Instructor: Swabha Swayamdipta

USC CSCI 444 NLP

Oct 15, 2025



Announcements + Logistics

- Today: HW2 due
- Next Mon: Flipped Classroom with Project Discussions
 - You work on your project in class! Every team takes turns to come chat with me regarding your proposal feedback, where you are feeling stuck, etc.
- Also Next Mon: HW3 released
- Next Wed: Quiz 3

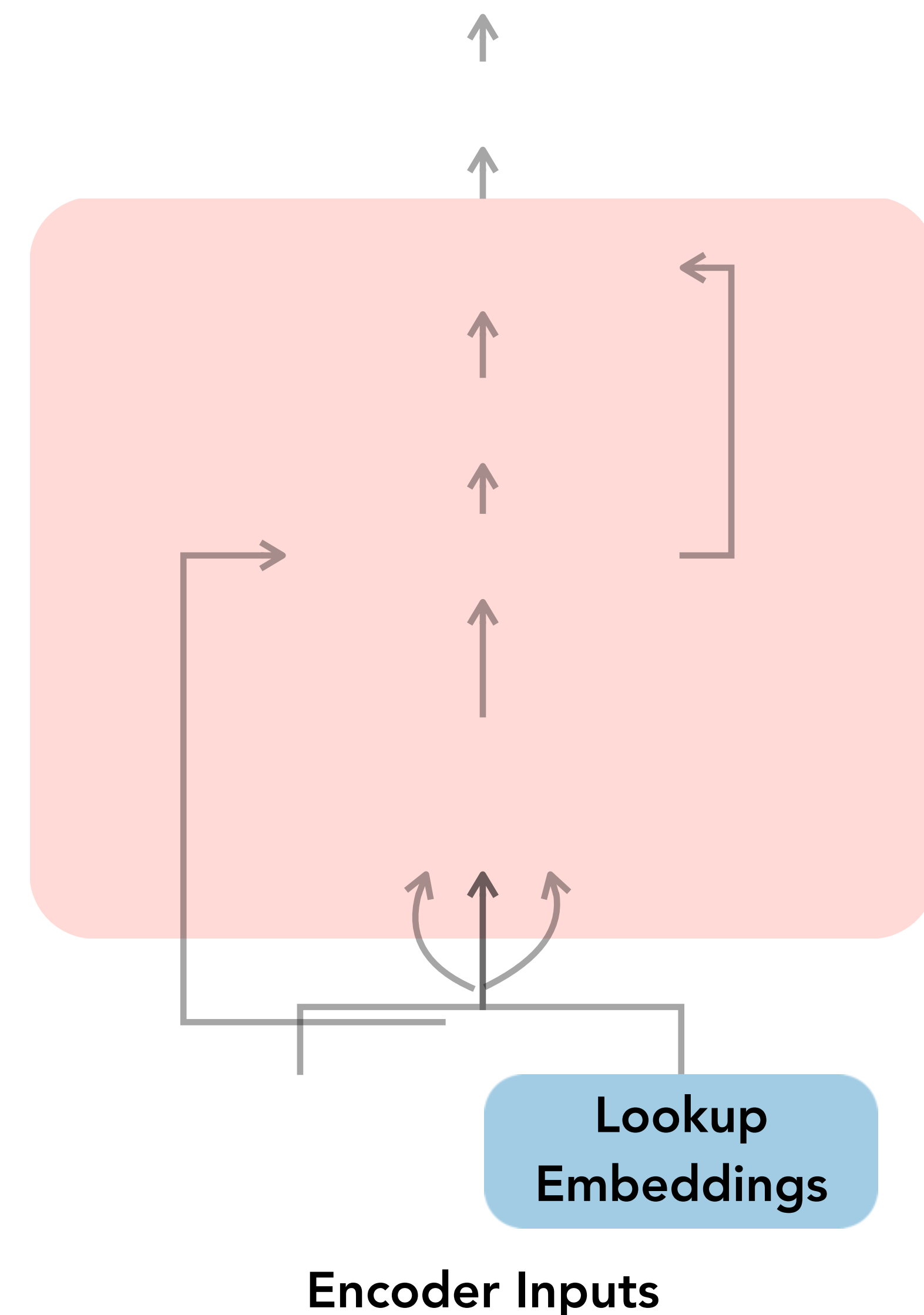
Lecture Outline

- Announcements
- Tokenization in modern LMs
- Natural Language Generation
 - Classic Inference Algorithms: Greedy and Beam Search
- Modern Generation: Sampling

Tokenization in Modern LMs

The Input Layer

- So far, we have made some assumptions about a language's vocabulary
- Our approach so far: use a known, fixed vocabulary
 - Built from training data, with tens of thousands of components
 - However, even with the largest vocabulary, we may encounter out-of-vocabulary words at test time
 - Our approach so far: map novel words seen at test time (OOV) to a single UNK



How to get the words?

Or, more accurately, the tokens?

- Problem: break the text into a sequence of discrete tokens
- For alphabetic languages such as English, deterministic scripts usually suffice to achieve accurate tokenization
- However, in languages such as Chinese and Swahili, words are typically composed of a small number of characters, without intervening whitespace

Word Structure in Language

- Finite vocabulary assumptions make even less sense in many languages.
 - Many languages exhibit complex morphology, or word structure.
 - The effect is more word types, each occurring fewer times.

-ambia = to tell

Example: Swahili verbs can have hundreds of conjugations, each encoding a wide variety of information. (Tense, mood, definiteness, negation, information about the object, ++)

Source: Wiktionary

Conjugation of -ambia																		
Form		Non-finite forms																
		Positive																
Infinitive		kuambia																
Positive form		Simple finite forms																
		Singular																
Imperative		ambia																
Habitual		huambia																
Plural		Complex finite forms																
		Classes																
Polarity	Persons				Persons / Classes													
	Sg.	1st Pl.	2nd Pl.	3rd Pl.	Sg. / 1st Pl.	2nd Pl.	3	4	5	6	7	8	9	10	11 / 14	15 / 17	16	18
Past																		
Positive	niliambia	tuliambia	uliambia	mliambia	aliambia	waliambia	uliambia	iliambia	liliambia	yaliambia	kiliambia	viliambia	iliambia	ziliambia	uliambia	kuliambia	paliambia	muliambia
Negative	sikuambia	hatukuambia	hukuambia	hamkuambia	hakuambia	hawakuambia	haukuambia	haikuambia	halikuambia	hayakuambia	hakikuambia	havikuambia	haikuambia	hazikuambia	haukuambia	hakukuambia	hapakuambia	hamukuambia
Present																		
Positive	ninaambia	tunaambia	unaambia	mnaambia	anaambia	wanaambia	unaambia	inaambia	linaambia	yanaambia	kinaambia	vinaambia	inaambia	zinaambia	unaambia	kunaambia	panaambia	munaambia
Negative	siambia	hatuambia	huambia	hamambia	haambia	hawaambia	hauambia	haiambia	haliambia	hayaambia	hakiambia	haviambia	haiambia	haziambia	hauambia	hakuambia	hapaambia	hamuambia
Future																		
Positive	nitaambia	tutaambia	utaambia	mtaambia	ataambia	wataambia	utaambia	itaambia	litaambia	yataambia	kitaambia	vitaambia	itaambia	zitaambia	utaambia	kutaambia	pataambia	mutaambia
Negative	sitaambia	hatutaambia	hutaambia	hamtaambia	hataambia	hawataambia	hautaambia	haitaambia	halitaambia	hayataambia	hakitaambia	havitaambia	haitaambia	hazitaambia	hautaambia	hakutaambia	hapataambia	hamutaambia
Subjunctive																		
Positive	niambia	tuambia	uambia	mambia	aambia	waambia	uambia	iambia	liambia	yaambia	kiambia	viambia	iambia	ziambia	uambia	kuambia	paambia	muambia
Negative	nisiambia	tusiambia	usiambia	msiambia	asiambia	wasiambia	usiambia	isiambia	lisiambia	yasiambia	kisiambia	visiambia	isiambia	zisiambia	usiambia	kusiambia	pasiambia	musiambia
Present Conditional																		
Positive	ningeambia	tungeambia	ungeambia	mngeambia	angeambia	wangeambia	ungeambia	ingeambia	lingeambia	yangeambia	kingeambia	vingeambia	ingeambia	zingeambia	ungeambia	kungeambia	pangeambia	mungeambia
Negative	nisingeambia	tusingeambia	usingeambia	msingeambia	asingeambia	wasingeambia	usingeambia	isingeambia	lisingeambia	yasingeambia	kisingeambia	visingeambia	isingeambia	zisingeambia	usingeambia	kusingeambia	pasingeambia	musingeambia
Past Conditional																		
Positive	ningaliambia	tungaliambia	ungaliambia	mngaliambia	angaliambia	wangaliambia	ungaliambia	ingaliambia	lingaliambia	yangaliambia	kingaliambia	vingaliambia	ingaliambia	zingaliambia	ungaliambia	kungaliambia	pangaliambia	mungaliambia
Negative	nisingaliambia	tusingaliambia	usingaliambia	msingaliambia	asingaliambia	wasingaliambia	usingaliambia	isingaliambia	lisingaliambia	yasingaliambia	kisingaliambia	visingaliambia	isingaliambia	zisingaliambia	usingaliambia	kusingaliambia	pasingaliambia	musingaliambia
Conditional Contrary to Fact																		
Positive	ningeliambia	tungeliambia	ungeliambia	mngeliambia	angeliambia	wangeliambia	ungeliambia	ingeliambia	lingeliambia	yangeliambia	kingeliambia	vingeliambia	ingeliambia	zingeliambia	ungeliambia	kungeliambia	pangeliambia	mungeliambia
Gnomic																		
Positive	naambia	twaambia	waambia	mwaambia	aambia	waambia	waambia	yaambia	laambia	yaambia	chaambia	vyaambia	yaambia	zaambia	waambia	kwaambia	paambia	mwaambia
Perfect																		

Subword Modeling

- Solution: look at subwords!
- Subword modeling encompasses a wide range of methods for reasoning about structure below the word level
 - Subwords may be words, parts of words, characters, bytes
- The dominant modern paradigm is to learn a vocabulary of parts of words (subword tokens)
- At training and testing time, each word is split into a sequence of known subwords
- Different algorithms:
 - Byte-Pair Encoding
 - WordPiece Modeling
 - Follow different strategies. Often contain prepending / appending special tokens (##, </w>)

Hello how are U tday?



hello how are u tday?



[hello, how, are, u, tday, ?]



[hello, how, are, u, td, ##ay, ?]

Word structure and subword models

- Common words end up being a part of the subword vocabulary, while rarer words are split into (sometimes intuitive, sometimes not) components.
- In the worst case, words are split into as many subwords as they have characters.
 - Llama 3 uses a tokenizer with a vocabulary of 128K tokens

	word		vocab mapping	embedding
Common words	hat	→	hat	
	learn	→	learn	
Variations	taaaaasty	→	taa## aaa## sty	
misspellings	laern	→	la## ern##	
novel items	Transformerify	→	Transformer## ify	

Byte-pair encoding

- Byte-pair encoding is a simple, effective strategy for defining a subword vocabulary
- Adapted for word segmentation from data compression technique (Gage, 1994)
 - Instead of merging frequent pairs of bytes, we merge characters or character sequences
- Algorithm:
 1. Start with a vocabulary containing only characters and an “end-of-word” symbol.
 2. Using a corpus of text, find *the most common adjacent characters* “a,b”; add “ab” as a subword
 - This is a learned operation! However, not a parametric function
 - Only combine pairs (hence the name!)
 3. Replace instances of the character pair with the new subword; repeat until desired vocabulary size.
- At test time, first split words into sequences of characters, then apply the learned operations to merge the characters into larger, known symbols
- Originally used in NLP for machine translation; now a similar method (WordPiece) is used in pretrained models.

BPE in action

Corpus		
low	lower	newest
low	lower	newest
low	widest	newest
low	widest	newest
low	widest	newest

Corpus		
low</w>	lower</w>	newest</w>
low</w>	lower</w>	newest</w>
low</w>	widest</w>	newest</w>
low</w>	widest</w>	newest</w>
low</w>	widest</w>	newest</w>

Corpus		
l o w </w>	l o w e r </w>	n e w e s t </w>
l o w </w>	l o w e r </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>

Vocabulary								
d	e	i	l	n	o	s	t	w
es								

Frequency		
d-e (3)	l-o (7)	t-</w> (8)
e-r (2)	n-e (5)	w-</w> (5)
e-s (8)	o-w (7)	w-e (7)
e-w (5)	r-</w> (2)	w-i (3)
i-d (3)	s-t (8)	

BPE in action

Corpus		
low	lower	newest
low	lower	newest
low	widest	newest
low	widest	newest
low	widest	newest

Corpus		
low</w>	lower</w>	newest</w>
low</w>	lower</w>	newest</w>
low</w>	widest</w>	newest</w>
low</w>	widest</w>	newest</w>
low</w>	widest</w>	newest</w>

Corpus		
l o w </w>	l o w e r </w>	n e w e s t </w>
l o w </w>	l o w e r </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>

Vocabulary								
d	e	i	l	n	o	s	t	w
es	est							

Frequency		
d-es (3)	l-o (7)	w-</w> (5)
e-r (2)	n-e (5)	w-es (5)
e-w (5)	o-w (7)	w-e (2)
es-t (8)	r-</w> (2)	w-i (3)
i-d (3)	t-</w> (8)	

BPE in action

Corpus		
low	lower	newest
low	lower	newest
low	widest	newest
low	widest	newest
low	widest	newest

Corpus		
low</w>	lower</w>	newest</w>
low</w>	lower</w>	newest</w>
low</w>	widest</w>	newest</w>
low</w>	widest</w>	newest</w>
low</w>	widest</w>	newest</w>

Corpus		
l o w </w>	l o w e r </w>	n e w e s t </w>
l o w </w>	l o w e r </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>
l o w </w>	w i d e s t </w>	n e w e s t </w>

Vocabulary								
d	e	i	l	n	o	s	t	w
es	est	est</w>	lo	low	low</w>	ne	new	newest</w>

After 10 merges

WordPiece Modeling

- Algorithm from Google, similar to BPE
- Identifies subwords by adding a prefix (##)
 - Each word is initially split by adding ## to all the characters inside a word
 - So, for instance, "word" gets split like this: w ##o ##r ##d
 - For this vocabulary:
 - ("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
 - Splits may look like:
 - ("h" "##u" "##g", 10), ("p" "##u" "##g", 5), ("p" "##u" "##n", 12), ("b" "##u" "##n", 4), ("h" "##u" "##g" "##s", 5)
- On merging, ## **between** the two tokens is removed
 - This explains the presence of the token "##ing"

WordPiece Modeling Outcome

- Different stopping criteria: number of merges or size of resulting vocabulary
- In the worst case, at test time, words are split into as many subwords as they have characters
- Common words end up being a part of the subword vocabulary, while rarer words are split into (sometimes intuitive, sometimes not) components

WordPiece Outcome

	word		vocab mapping	embedding
Common words	hat	→	hat	
	learn	→	learn	
Variations	taaaaasty	→	taa## aaa## sty	  
misspellings	laern	→	la## ern##	 
novel items	Transformerify	→	Transformer## ify	 

Tokenization: Frequently Asked Questions

- Where does the token “##ing” come from?
 - In WordPiece tokenization, all non-starting characters are initialized as ##x.
 - Like: h, ##e, ##l, ##l, ##o.
 - Upon merging, only the first segment keeps its ##.
- How is tokenization done in Chinese?
 - Follows the same broad overall algorithm, but the initial split into characters involve language-specific rules
 - e.g. stroke-level tokenization [Si et al., 2023]

Source: <https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt>